



Arquitectura Actual (AS-IS), Análisis Técnico, Riesgos e Inventario

Modernización y Contenerización POS

MARZO 2026

1. Objetivo y Alcance

El presente documento establece una línea base técnica exhaustiva (Estado AS-IS) del ecosistema de aplicaciones empresariales de EDIMCA. El alcance abarca la plataforma de Punto de Venta (POS), Control de Producción, Sincronización con el ERP (JD Edwards), servicios de gestión documental (OCR, PDF) y componentes transversales (Gateway, Autenticación, DTO, Administración).

El objetivo principal es:

- Identificar la arquitectura real actual y sus patrones de despliegue.
- Inventariar de manera cruzada servidores, componentes, puertos y tecnologías.
- Evidenciar de forma determinista la obsolescencia, vulnerabilidades y deuda técnica.
- Establecer una base técnica inmutable para la planificación de la modernización (TO-BE).

2. Contexto del Ecosistema EDIMCA

El ecosistema evaluado constituye el núcleo transaccional del negocio, soportando:

- Ventas en tienda (POS).
- Control de producción y transformaciones en bodega.
- Sincronización bidireccional con el ERP JD Edwards.
- Gestión documental automatizada (generación de PDF y lectura óptica OCR).
- Integración asincrónica de eventos mediante mensajería JMS.

Características Arquitectónicas Clave:

Se trata de un sistema altamente distribuido pero **fuertemente acoplado**. Presenta una mezcla de tecnologías *legacy* (Java EE, EJB, React 16, Node 10) con implementaciones modernas aisladas (Spring Boot 3, Angular 13), sosteniéndose sobre una infraestructura *on-premise* con dependencias transversales críticas (Librerías DTO compartidas).

3. Ambientes Identificados

La configuración de los repositorios (application-dev, qa, prod) confirma la existencia lógica de tres ambientes:

Ambiente	Descripción	Estado Operativo
DEV	Desarrollo	Activo
QA	Certificación / Calidad	Activo
PROD	Producción	Activo

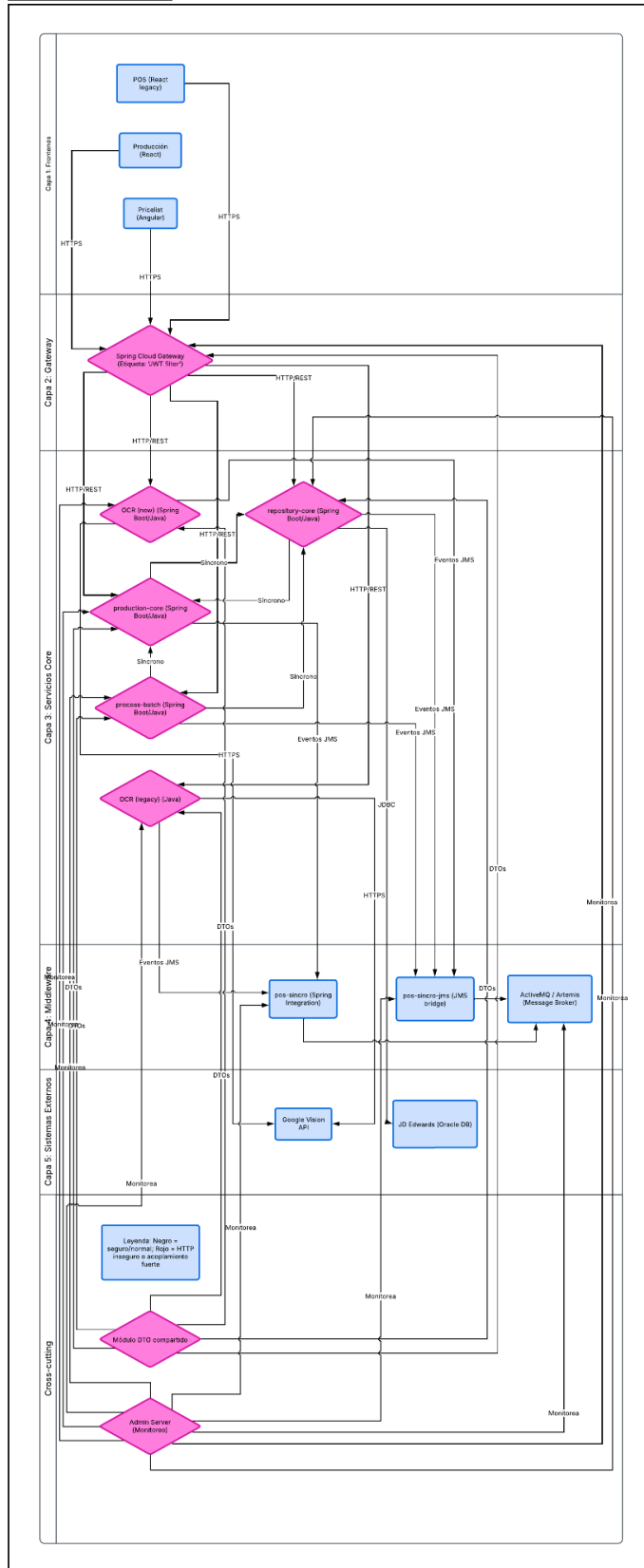
Problema Estructural Detectado: No existe una estandarización real ni aislamiento estricto entre ambientes. Se evidencia el uso de URLs y puertos hardcodeados, credenciales embebidas en los archivos .properties y dependencias directas a localhost. La matriz de despliegue (Anexo A2) confirma que QA y PROD comparten nodos físicos/virtuales en ciertos servicios.

4. Arquitectura General Actual (AS-IS)

La topología lógica del ecosistema se divide en 7 capas funcionales:

1. **Capa de Canales (Frontend):** Aplicaciones React y Angular consumiendo servicios vía HTTP estático.
2. **Capa de Enrutamiento (Gateway):** Spring Cloud Gateway actuando como proxy y filtro JWT (inseguro).
3. **Capa de Servicios Core (Backend):** Múltiples servicios independientes (Spring Boot) sin estándares comunes de observabilidad.
4. **Capa de Middleware:** Bus de eventos JMS (ActiveMQ/Artemis) y adaptadores SOAP.
5. **Capa de Sistemas Externos (ERP):** JD Edwards (Oracle BSSV).
6. **Capa de Servicios Auxiliares:** Utilidades aisladas para OCR y generación de PDF.
7. **Capa de Datos:** PostgreSQL (Maestro/Réplica con PGPool) y Redis.

CA Architecture Diagram - Detailed Container Level



5. Flujos Transaccionales Críticos

Flujo de Negocio	Descripción del Flujo	Nivel de Riesgo Operativo
POS → Core	Ejecución de ventas, recaudos y órdenes de producción.	● Crítico
Core → JDE	Integración SOAP/BSSV para sincronización de maestros y transacciones.	● Crítico
Eventos JMS	Desacople asíncrono y propagación de estados (órdenes, pagos).	● Crítico
Motor Batch	Cierres de caja y sincronización masiva de listas de precio.	● Crítico
Lectura OCR	Extracción automatizada de montos en comprobantes.	● Medio

6. Hallazgos Transversales (Clave)

6.1. Seguridad Comprometida

- Filtros de JWT inválidos o deficientes (tokens sin tiempo de expiración).
- Secretos, credenciales de base de datos y llaves de APIs hardcodeados en el código.
- Políticas de CORS abiertas (*) exponiendo los servicios a cualquier origen.
- Endpoints de administración (Spring Boot Actuator) expuestos sin autenticación.

6.2. Arquitectura Rígida

- Acoplamiento fuerte debido a un módulo DTO centralizado que obliga a despliegues en cascada.
- Duplicidad de servicios (coexistencia de un OCR Legacy y uno Nuevo).
- Ausencia de versionado de contratos en las APIs.

6.3. Operación y Calidad Deficiente

- Inexistencia de pipelines CI/CD automatizados y estandarizados para los backends.
- Ausencia de observabilidad real (uso de System.out.println y printStackTrace).
- Carencia total de pruebas unitarias o de integración en los repositorios revisados.
- Manejo de errores deficiente con retornos HTTP 200 ante fallos funcionales.

ANEXOS TÉCNICOS DETALLADOS

ANEXO A1. Inventario Completo de Componentes (Nivel Técnico)

Componente	Descripción Funcional	Tecnología / Stack Base	Módulos / Clases Principales	Observaciones Técnicas Críticas
edimca-pos-front	App principal POS (ventas, recaudos, órdenes).	React 16.13, MobX 5, CRA 3.1.1, Node 12	App.js, Routes.js, services/, store/	● React 16 obsoleto, CRA deprecado. Múltiples CVEs en librerías (axios, xlsx, jspdf). Configuración de red hardcodeada.
edimca-producti on-front	Interfaz de control de producción en bodega.	React 17, PrimeReact 6, CRA 3.4.1	views/productio n, services/api.js, utils/	● Stack legacy. Lógica funcional hardcodeada (ej. sucursales MCO). Manejo de errores silencioso.
pricelist-project	Gestión de lista de precios y aprobaciones.	Angular 13, PrimeNG 13, TypeScript 4.4	modules/pricelist , services/, auth/	● Angular 13 EOL. Token almacenado en sessionStorage. Uso de comunicaciones HTTP sin TLS.
edimca-producti on-core	Backend lógica de producción y operaciones POS.	Java 11, Spring Boot 2.1.5, Jersey 2.26	controller/, service/, repository/	● Spring Boot EOL. Credenciales hardcodeadas. Uso de SQL nativo concatenado. Alto acoplamiento.
edimca-process-	Sincronización	Java 11, Spring	batch/, jobs/,	● Actuator

batch	batch con JDE (stock, cierres).	Boot 2.7.6	scheduler/	expuesto públicamente. Errores lógicos en condicionales (`
edimca-repository-core	Servicio de gestión documental y recursos.	Java 11, Spring Boot 2.1.5	FileController, FileService, Utils	● Sin autenticación. Riesgo severo de <i>Path Traversal</i> . Uso restrictivo de rutas locales (Windows).
edimca-pos-sincro	Middleware de sincronización POS ↔ JDE (SOAP).	Java EE (JAX-WS), Jersey, Payara/GlassFish	SyncService, ClientJDE, Threads	● Uso de threads manuales (Thread.sleep). Mezcla de compilación Java 8/11. Fuerte dependencia del app server.
edimca-pos-sincro-jms	Eventos asincrónicos POS mediante mensajería.	Java EE + Spring Boot 2.4.5, ActiveMQ 5.x	JMSListener, Producer, Config	● CVE crítico en ActiveMQ. Credenciales hardcodeadas en texto plano. Conflicto de namespaces javax/jakarta.
edimca-get-amount-pd	OCR moderno para extracción de montos (Cloud).	Java 17, Spring Boot 3.5.7, Google Vision API	OCRController, VisionService, Parser	● Moderno pero carente de seguridad perimetral. Secretos expuestos en JSON. Nula cobertura de pruebas.
edimca-get-amount-old	OCR legacy basado en	Java 11, Quarkus 2.x, Tess4J	OCRService, ImageProcessor	● Código monolítico (+600

	Tesseract local.			líneas). Dependiente de librerías del SO. Sin pruebas automatizadas.
edimca-read-documents	Servicio auxiliar OCR de imágenes.	Python 3.11, Flask, OpenCV, Tesseract	app.py, ocr.py, utils/	🟡 Dependencia de filesystem local (/home/...). Inconsistencias en archivo requirements.txt.
pdf-project	Generación de PDFs transaccionales desde HTML.	Node.js 10, Express 4.16.3, Puppeteer 1.20	server.js, pdfService.js	🔴 Node 10 EOL absoluto. Puppeteer inseguro (--no-sandbox). CVEs confirmados en dependencias de Express.
edimca-gateway-service	API Gateway central (Enrutamiento y seguridad).	Java 11, Spring Boot 2.6.10, SC Gateway	GatewayConfig, JwtFilter	🔴 Implementación JWT insegura. Políticas CORS abiertas. Actuador expuesto a la red.
auth-project	Autenticación y generación de tokens JWT.	Java 11, Spring Boot 2.6.10	AuthController, JwtUtil	🔴 Token generado sin tiempo de expiración. Secreto de firma hardcodeado. Dependencia de red a localhost.
edimca-core-dto	Librería compartida (DTOs, enums, utilitarios).	Java 11, Spring Boot 2.1.5	Dto*, Enums, Utils	🔴 Crítico Sistémico: Secretos de negocio en

				enumeradores. Crypto inseguro (MD5). Genera acoplamiento monolítico.
adminconsole	Monitoreo basado en Spring Boot Admin.	Java 11, Spring Boot Admin 2.6	AdminServerApp lication	🟡 Sin seguridad perimetral visible. Carece de configuración externalizada por ambiente.
edimca-pos-core	Núcleo POS (Orquesta operaciones de negocio).	Java 11, Spring Boot	OrderService, PaymentService, Sync	🔴 Dependencia central. Alto acoplamiento funcional que bloquea cualquier refactorización ágil.

ANEXO A2. Matriz Repositorio ↔ Componente ↔ Servidor ↔ Puerto ↔ Ambiente

Notas de interpretación:

- [✓] **Confirmado:** Evidencia explícita en código o configuración cruzada con infraestructura.
- [△] **Inferido:** Asignación deducida por rol o puerto; requiere validación física al faltar declaración en código.

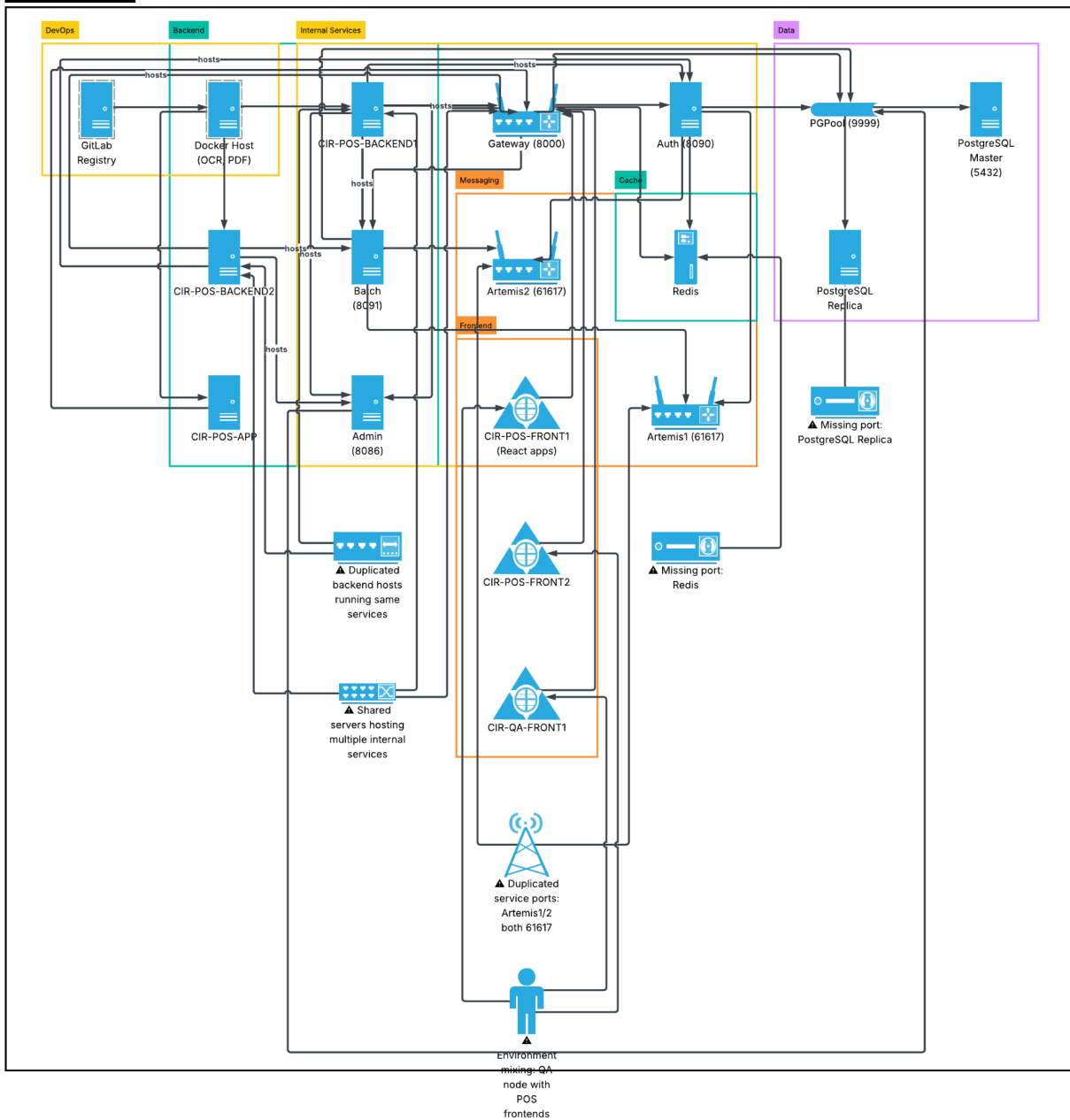
Repositorio / Proyecto	Componente Lógico	Servidor (Hostname)	IP Asignada	Puerto	Entorno	Evidencia de Trazabilidad
edimca-pos-f rontr-pd149	POS Frontend	CIR-POS-F RONT1	172.16.14 9.113	5000	PROD	✓ Configuración de servidor
edimca-pos-f rontr-pd149	POS Frontend	CIR-POS-F RONT2	172.16.14 9.114	5000	PROD	✓ Nodo HA frontend
edimca-pos-f rontr-pd149	POS Frontend	CIR-QA-FR ONT1	172.16.14 9.116	5000	QA	✓ Nodo de certificación
edimca-prod uction-front- pd149	CP Frontend	CIR-POS-F RONT1	172.16.14 9.113	5000	PROD	△ Comparte server frontend
edimca-prod uction-front- pd149	CP Frontend	CIR-POS-F RONT2	172.16.14 9.114	5000	PROD	△ Comparte nodo HA
pricelist-proj ect-pd149	PriceList Frontend	CIR-POS-A PP	172.16.14 9.163	4200	PROD	✓ Despliegue Angular
edimca-prod uction-core-p d149	Production Core	CIR-POS-B ACKEND1	172.16.14 9.117	8080	PROD	✓ Backend principal
edimca-prod uction-core-p d149	Production Core	CIR-POS-B ACKEND2	172.16.14 9.118	8080	PROD	✓ Nodo HA backend
edimca-proce ss-batch-pd1 49	Batch	CIR-POS-B ACKEND2	172.16.14 9.118	8091	PROD	✓ Definido explícitamente
edimca-gate way-service- pd149	Gateway	CIR-POS-A PP	172.16.14 9.163	8000	PROD	✓ Definido explícitamente

auth-project-pd149	Auth Service	CIR-POS-B ACKEND2	172.16.14 9.118	8090	PROD	✓ Definido explícitamente
adminconsole	Admin Server	CIR-POS-B ACKEND2	172.16.14 9.118	8086	PROD	✓ Definido explícitamente
edimca-pos-s incro-pd149	POS Sync	CIR-POS-B ACKEND1	172.16.14 9.117	8080	PROD	△ Comparte backend
edimca-pos-s incro-jms-pd 149	POS Sync JMS	CIR-Atermis 1	172.16.14 9.108	61617	PROD	✓ Nodo Artemis MQ
edimca-pos-s incro-jms-pd 149	POS Sync JMS	CIR-Atermis 2	172.16.14 9.109	61617	PROD	✓ HA Artemis MQ
edimca-repos itory-core-pd	Repository Service	CIR-MOD-P ROY	172.16.14 9.164	8080	PROD/QA	✓ Definido explícitamente
edimca-get-a mount-pd	OCR nuevo	CIR-POS-D K-CONT	172.16.14 9.169	50001	PROD/QA	✓ Definido explícitamente
edimca-get-a mount-old	OCR legacy	CIR-POS-D K-CONT	172.16.14 9.169	N/D	PROD	△ Puerto no definido
pdf-project- master	PDF Service	CIR-POS-D K-CONT	172.16.14 9.169	N/D	PROD	△ Asignación inferida
edimca-read- documents	OCR Python	CIR-POS-D K-CONT	172.16.14 9.169	N/D	PROD	△ Asignación inferida
PostgreSQL	DB Master	CIR-pgsql	172.16.14 9.103	5432	PROD	✓ Servicio de DB
PostgreSQL	DB Replica	CIRREP-BD POS	172.16.14 9.173	5432	PROD	✓ Réplica DB
PostgreSQL PGPool	PGPool	CIR-PGPO OL	172.16.14 9.174	9999	PROD	✓ Pool de conexiones
Redis	Cache	CIR-POS-R EDIS	172.16.14 9.140	N/D	PROD	✓ Memoria Caché
GitLab Registry	DevOps	CIR-POS-D K-REG	172.16.14 9.168	N/D	PROD/QA	✓ Registro Contenedores

Hallazgos Críticos de la Matriz de Despliegue

- **1. Colisión de servicios en mismos nodos:** Múltiples servicios críticos (Core, Batch, Auth, Admin) comparten el mismo servidor CIR-POS-BACKEND2 sin aislamiento lógico, propiciando fallos en cascada por consumo de recursos.
- **2. Falta de puertos documentados:** Los utilitarios contenerizados (OCR Legacy, PDF, OCR Python) carecen de puertos explícitos. Esto bloquea la creación de reglas de firewall, observabilidad y migración.
- **3. Mezcla de ambientes:** Se observa que recursos como CIR-MOD-PROY y CIR-POS-DK-CONT operan cargas combinadas de PROD y QA, violando políticas de seguridad.
- **4. Acoplamiento fuerte de red:** Los Frontends apuntan a IPs de Backends fijos, y estos a la BD directamente, careciendo de Service Discovery.

Deployment Architecture Diagram



ANEXO A3. Matriz Completa de Obsolescencia

● Sistemas Operativos

Componente	Tecnología	Estado del Ciclo de Vida	Detalle / Riesgo
Servidores Backend	RHEL 7	EOL	Sin soporte de seguridad ni parches del fabricante.
Servidores Frontend	Linux (Nginx)	Medio	Dependencia manual de configuraciones del SO.
Nodos Docker	RHEL 9.6	OK	Sistema Operativo moderno y soportado.

● Backend

Componente	Tecnología	Estado del Ciclo de Vida	Detalle / Riesgo
production-core	Spring Boot 2.1	EOL	Versión de framework completamente sin soporte.
repository-core	Spring Boot 2.1	EOL	Vulnerabilidades no parcheables.
batch	Spring Boot 2.7	Legacy	Fin del soporte Open Source (OSS).

● Frontend

Componente	Tecnología	Estado del Ciclo de Vida	Detalle / Riesgo
POS	React 16	EOL	Dependencias vulnerables.
Production	React 17	Legacy	Sin soporte extendido activo.
Pricelist	Angular 13	EOL	Riesgo medio-alto por componentes desactualizados.

● Middleware y Utilitarios

Componente	Tecnología	Estado del Ciclo de Vida	Detalle / Riesgo
JMS Broker	ActiveMQ 5.x	Vulnerable	CVE crítico documentado.
Sincro	Java EE	Legacy	Fuerte dependencia de servidor de aplicaciones on-premise.
PDF	Node.js 10	EOL Absoluto	Riesgo Crítico en el motor de renderizado.
OCR Old	Quarkus 2	Legacy	Reemplazable por el nuevo motor OCR.

ANEXO A4. Inventario de Vulnerabilidades

Vulnerabilidad	Clasificación	Componente Afectado	Detalle Técnico	Nivel de Impacto
JWT sin expiración	Seguridad	Gateway / Auth	Tokens de acceso válidos indefinidamente, riesgo de secuestro.	Crítico
Secretos Hardcodeados	Seguridad	Core DTO, Backend	Contraseñas de DB y correos visibles en propiedades y código.	Crítico
CORS Abierto (*)	Seguridad	Múltiples servicios	Permite acceso no controlado a APIs desde cualquier dominio.	Alto
Actuator Expuesto	Seguridad	Batch / Gateway	Exposición total de métricas internas y configuración del sistema.	Alto
CVE ActiveMQ	Seguridad	POS Sincro JMS	Vulnerabilidad CVE-2023-46604 (RCE) no parcheada.	Crítico
CVE Gson / Axios	Seguridad	Auth / POS Front	Vulnerabilidades de alto impacto en librerías transaccionales.	Alto
MD5 / DESede	Seguridad	Core DTO	Uso de criptografía deprecada y fácilmente reversible.	Crítico
SQL Concatenado	Seguridad	Production Core	Consultas nativas sin parametrización (Riesgo de SQL Injection).	Alto
Node.js 10	Plataforma	PDF Project	Runtime totalmente vulnerable a ataques de día cero.	Crítico
Dependencia Filesystem	Arquitectura	Múltiples	Acceso directo a rutas de SO vulnerando principios de aislamiento.	Alto
Sin Autenticación	Seguridad	Repository Core	Endpoints expuestos a la red interna sin validación de identidad.	Crítico

ANEXO A5. Matriz de Dependencias (Completas)

Dependencias Externas (Hacia el Ecosistema)

Sistema	Tipo	Uso / Contexto
JD Edwards (Oracle)	ERP	Integración SOAP/BSSV para el núcleo transaccional.
Google Vision API	API Cloud	Integración REST para el nuevo motor de extracción OCR.
Servidor SMTP	Infraestructura	Envío de notificaciones y correos de la operación.
Servidor SFTP	Infraestructura	Transferencia de archivos seguros.
Filesystem Local	SO Host	Escritura transitoria y definitiva de PDFs y adjuntos.

Dependencias Internas Críticas

Componente Solicitante	Depende de	Tipo de Dependencia
Gateway	Auth	Validación de seguridad y ruteo de peticiones.
Frontend (React/Angular)	Backends REST	Integración de datos vía HTTP.
Core Services	Core DTO	Contratos de datos, enums y llaves compartidas.
Motor Batch	BD + Core	Acceso directo a bases de datos y llamadas HTTP loopback.
Utilitario OCR/PDF	Backend	Ejecución como servicio auxiliar interno.

👉 **Problema Crítico Identificado:** La librería compartida **edimca-core-dto** genera un acoplamiento global masivo. La imposibilidad de versionarlo dinámicamente genera un impacto en cascada, obligando a redespugar múltiples aplicaciones ante un cambio mínimo.

ANEXO A6. Problemas de Portabilidad (Bloqueantes Cloud)

Problema Estructural	Detalle Técnico	Impacto en Modernización
Rutas Absolutas	Referencias a /home/..., C:/Users/...	Rompe la ejecución de contenedores Docker al no encontrar discos fijos.
IPs Hardcodeadas	Endpoints fijos en Config.js / .properties	Aplicación no portable entre entornos sin recompilación de código.
Dependencias localhost	Servicios llamándose a sí mismos en 127.0.0.1	Falla garantizada en orquestadores modernos (ej. Kubernetes).
Filesystem Compartido	Lógica basada en persistencia en disco local	Impide la escalabilidad horizontal (Múltiples instancias).
Config. Embebida	Falta de inyección de configuración externa	Obliga a despliegues manuales y artefactos no inmutables.
Dependencias de SO	Tesseract y binarios instalados en el host	Bloquea la adopción de arquitecturas Cloud Native puras.

ANEXO A7. Evaluación de Modernización por Componente

Componente	Acción Sugerida	Descripción de la Estrategia
Gateway	Refactor Urgente	Implementar seguridad real (Filtro JWT válido, <i>Rate Limiting</i>).
Auth	Rediseño Total	Migrar el modelo hacia estándares modernos (OAuth2 / Keycloak).
Core DTO	Rediseño Total	Eliminar acoplamiento. Extraer a micro-librerías con contratos versionados.
Production Core	Refactor	Modernización de framework hacia Spring Boot 3.
Process Batch	Refactor	Aislar lógica, eliminar loopback local y mejorar resiliencia del job.
OCR Nuevo	Hardening	Aplicar seguridad perimetral y cobertura de pruebas.
OCR Viejo	Retiro (Decommission)	Eliminar dependencia técnica tras certificar la funcionalidad del nuevo.
Frontend POS	Modernizar	Planificar migración fuera de React 16.
Pricelist	Upgrade	Subir a versión de Angular LTS actual.
Repository Core	Refactor	Abstraer sistema de archivos e implementar seguridad de endpoint.
PDF Project	Reemplazo	Reescribir en Node moderno o delegar a servicio nativo de reportes.

CONCLUSIÓN ESTRATÉGICA Y EJECUTIVA

El ecosistema EDIMCA, si bien cumple funcionalmente con el negocio, presenta un nivel significativo de **deuda técnica acumulada** con impacto directo y limitante en cuatro vectores:

- **Seguridad:** Múltiples vulnerabilidades críticas activas (JWT defectuoso, secretos hardcodeados, CORS irrestricto), exposición no autorizada de endpoints internos y uso de librerías base con CVEs explotables.
- **Obsolescencia:** La operación descansa sobre infraestructura y lenguajes en EOL (RHEL 7, Spring Boot 2.x, React 16, Node 10). Esto no solo implica una carencia absoluta de soporte, sino que limita radicalmente la capacidad técnica de innovación.
- **Arquitectura:** El alto acoplamiento entre servicios dictado por el DTO compartido, la duplicación funcional y la falta de contratos de API versionados hacen que el ecosistema sea frágil ante los cambios (Baja tolerancia a fallos).
- **Operación:** La ausencia de observabilidad real, el despliegue manual dependiente del filesystem local y la inexistencia de pipelines CI/CD resultan en tiempos de recuperación lentos ante incidentes.

Viabilidad de Modernización

El sistema en su estado actual **NO es Cloud-Ready** ni apto para un *Lift & Shift* directo a contenedores. Sin embargo, existen **oportunidades** claras de mejora debido a su arquitectura parcialmente modular, lo que permite una **migración incremental (Patrón Strangler)** priorizando la seguridad y la estandarización.

Ejes de Acción Inmediata:

1. **Seguridad Inmediata (Bloqueante):** Refactor de Autenticación, CORS y gestión de configuración externa.
2. **Desacoplamiento Arquitectónico:** Resolución de dependencias locales (IPs y Filesystem).
3. **Estandarización de Plataforma:** Iniciar la transición del stack operativo hacia contenedores modernizados.